

Section 13.1: Classic System Design Trade-Offs & Patterns

Comprehensive Interview Study Guide — 1 of 1 Files

How to use this file: Each sub-section follows a consistent structure: *What It Is* → *Core Mechanics* → *Trade-Off Matrix* → *When / Where / How to Use* → *Interview Design Problems with Solutions*.

Executive Overview

Every architectural decision is a trade-off. This section captures the fundamental tensions that arise in system design interviews: consistency vs. availability, latency vs. throughput, cost vs. reliability. Mastering these trade-offs demonstrates senior-level thinking — not by memorising facts, but by being able to reason aloud about *why* each choice is appropriate given the constraints at hand.

13.1.1 — Vertical vs. Horizontal Scaling

What It Is

Vertical scaling (“scale up”) adds resources — CPU, RAM, disk — to a single machine. Horizontal scaling (“scale out”) adds more machines to distribute load. These represent fundamentally different architectural philosophies with downstream consequences for fault tolerance, cost, and operational complexity.

Core Mechanics

Vertical Scaling:

```
Single node upgrade path:  
CPU:  8 cores  → 32 cores  → 96 cores  
RAM:  32 GB   → 128 GB   → 768 GB  
SSD:  1 TB    → 4 TB     → 16 TB
```

Bottleneck removed by hardware upgrade.
Hard ceiling: largest available instance type.
Single point of failure remains.

Horizontal Scaling:

Distributed node growth:

Nodes: 1 → 10 → 100 → 10,000

Coordination overhead grows (network, consensus, data routing).

Benefits: fault isolation, elastic cost, theoretically unbounded scale.

Requires stateless or partitioned-state design.

Trade-Off Matrix

Aspect	Vertical	Horizontal
Complexity	Low — single machine	High — distributed coordination
Failure Isolation	Poor — single point of failure	Excellent — node failures are isolated
Cost Efficiency	Good up to cloud instance limits	Excellent at scale; pay for exactly what you need
Latency	Low — no network hops	Higher — inter-node communication adds RTT
Max Scale	~1000x single node ceiling	Planet scale (Google, AWS)
Stateful Workloads	Easy — shared memory	Hard — requires sharding or external state store
Operational Tooling	Minimal	Complex — service discovery, load balancing, health checks

When / Where / How to Use

Choose Vertical When: - The workload is inherently single-threaded or tightly coupled (e.g., a legacy RDBMS with complex joins). - Team is small and distributed systems expertise is limited. - Low-latency requirements prohibit network round-trips (< 1 ms SLAs). - Short-term fix while a horizontal architecture is being designed.

Choose Horizontal When: - Stateless services (API servers, web frontends): naturally horizontal — add nodes behind a load balancer. - You need fault tolerance — N+1 redundancy is free with horizontal nodes. - Cost must scale elastically with traffic (auto-scaling groups). - Long-term architecture — horizontal scales without re-architecture.

Hybrid (Most Production Systems): Right-size each tier independently: a single large RDS primary (vertical) + horizontal read replicas + horizontally-scaled API servers.

Interview Design Problems

Problem 1 — E-commerce Flash Sale

Scenario: Your checkout service handles 500 RPS normally. A flash sale will spike to 50,000 RPS for 10 minutes. How do you scale?

Solution: The checkout service is stateless (session data is in Redis). Therefore horizontal scaling is appropriate. Pre-warm an auto-scaling group to 100× normal capacity 5 minutes before the sale. Use a load balancer to distribute traffic. After the sale, scale back down within 15 minutes to control cost. The database is the bottleneck — vertically scale the primary write node ahead of time and add read replicas for product catalogue queries (which are read-heavy). Avoid horizontally scaling the write path at the database tier without introducing complexity (Vitess, CockroachDB) — not worth it for a 10-minute spike.

Key trade-off stated: Stateless tier → horizontal; stateful write tier → vertical + read replicas.

Problem 2 — Latency-Sensitive Trading Engine

Scenario: A financial trading engine must execute orders in $< 500 \mu\text{s}$ end-to-end. How do you scale as order volume grows?

Solution: Vertical scaling is the correct choice here. Horizontal scaling introduces network hops and coordination latency that violate the SLA. Scale up to the largest available bare-metal instance with CPU affinity and NUMA-aware memory allocation. Use kernel bypass networking (DPDK / RDMA) to eliminate OS overhead. Partition order books by ticker symbol across separate vertical machines — this is *sharding*, not true horizontal scaling, and avoids inter-node coordination within a single order book. Accept the single-point-of-failure risk within a shard; use an active-passive standby for failover (seconds, not milliseconds).

Key trade-off stated: Latency SLA forces vertical; SPOF is accepted and mitigated by standby.

Problem 3 — Global Video Streaming CDN

Scenario: Design the video delivery layer of a streaming service to handle 10 million concurrent viewers globally.

Solution: No single machine can serve 10M concurrent streams — horizontal scaling is mandatory. Deploy edge PoPs (Points of Presence) in 50+ regions (horizontally scaled CDN nodes). Each PoP is a cluster of horizontally-scaled cache servers. Origin servers (where raw video is stored) can be vertically scaled large instances behind a smaller pool of horizontally-scaled origin fetchers. Session state (playback position) lives in a distributed key-value store (DynamoDB / Redis Cluster), not on any individual node, enabling any PoP to serve any viewer.

Key trade-off stated: Geo-distribution mandates horizontal; stateless video delivery makes it clean; state is externalised.

13.1.2 — Microservices vs. Monolith

What It Is

Microservices decompose a system into independently deployable, loosely-coupled units, each owning its own data store. A monolith packages all functionality into a single deployable artefact backed by a shared database. The debate is fundamentally about *when complexity pays off*.

Core Mechanics

Monolith: - Single codebase, single build, single deployment unit. - In-process function calls — nanosecond latency, no serialisation overhead. - Shared schema — joins across domain boundaries are trivial. - Difficult to scale specific hot functions without scaling everything. - One bad deploy takes down the entire system.

Microservices: - Independent codebases, build pipelines, and release cycles per service. - Network calls between services (HTTP/gRPC) — millisecond latency, serialisation cost. - Database-per-service — cross-domain queries require API calls or event-driven data denormalisation. - Independent scaling of hot paths (e.g., search service without scaling checkout). - Distributed transactions require sagas or 2PC — complex failure modes.

Trade-Off Matrix

Factor	Monolith	Microservices
Development Velocity (early)	Fast — single codebase, no API contracts	Slow — API contracts, service stubs, distributed testing
Development Velocity (at scale)	Slow — merge conflicts, coupled deployments	Fast — teams deploy independently
Scaling Flexibility	Poor — all-or-nothing	Excellent — per-service auto-scaling

Factor	Monolith	Microservices
Operational Overhead	Low — one binary to deploy and monitor	High — service mesh, distributed tracing, per-service alerts
Debugging Complexity	Low — single process, single log stream	High — distributed traces required (Jaeger, Zipkin)
Deployment Risk	High per-deploy blast radius	Low per-service; higher aggregate risk
Data Consistency	Easy — ACID transactions across tables	Hard — eventual consistency, sagas for distributed transactions
Onboarding	Simple — one repo, one IDE session	Complex — understanding service topology

When / Where / How to Use

Choose Monolith When: - Team is fewer than 10 engineers — operational overhead exceeds benefit. - Single bounded domain — a simple CRUD app, an internal tool, a startup MVP. - Tight latency budget (< 10 ms) — network hops between services are unacceptable. - Rapid iteration is critical — shipping fast beats architectural purity in early stage. - Low scale requirements (< 1,000 RPS, simple read/write patterns).

Choose Microservices When: - Team exceeds 20 engineers — Conway's Law: org structure dictates system structure. - Clearly distinct bounded contexts with different scaling, availability, or team ownership requirements. - Technology diversity is needed — ML service in Python, API in Go, analytics in Rust. - Failure isolation is a hard requirement — payment processing must not take down search. - Compliance isolation — PII data in a separate service with independent access controls.

Migration Pattern — Strangler Fig:

```

Phase 1: Monolith serves all traffic
Phase 2: Extract Service A (high-value, low-coupling)
         Route /api/service-a → new microservice
         Monolith handles everything else
Phase 3: Extract Service B, C, ...
Phase 4: Decommission monolith shell

```

This avoids a “big bang” rewrite and allows validation of each extracted service before proceeding.

Interview Design Problems

Problem 1 — Should Twitter's Timeline Be a Microservice?

Scenario: You're designing Twitter's timeline feature. Tweet storage, user follow graph, and timeline fanout all currently live in a monolith. The timeline generation is becoming the bottleneck. What do you do?

Solution: Extract timeline generation as a dedicated microservice. This is the canonical strangler fig extraction: timeline is a hot path with distinct scaling requirements (read-heavy fanout vs. write-heavy tweet ingestion). The Timeline Service owns a pre-computed timeline cache (Redis Sorted Set per user). It subscribes to a tweet event stream (Kafka). On new tweet, it fans out to followers' caches (fan-out-on-write for users with < 10K followers; fan-out-on-read for celebrities). The monolith remains for auth, user profiles, and tweet ingestion — extract those later as teams grow. This gives independent scaling of the timeline read path without a full rewrite.

Key trade-off stated: Extract the hot path first; don't decompose prematurely across the whole system.

Problem 2 — Payment System Isolation

Scenario: Your e-commerce platform's payment processing occasionally takes down the product catalogue when under heavy load. How do you fix this architecturally?

Solution: Separate payment processing into an isolated microservice with its own database, its own connection pool, and its own deployment pipeline. This provides blast-radius isolation — a payment service outage (or a Stripe API timeout) no longer blocks catalogue reads. Introduce circuit breakers (Hystrix / Resilience4j) at the catalogue service so it degrades gracefully if payment is down. Use async communication for non-critical flows (order confirmation email) via a message queue (SQS/ Kafka) so the payment service doesn't block on downstream services. The latency cost of an extra network hop to the payment service is acceptable because payment is already user-interactive and not latency-critical at the millisecond level.

Key trade-off stated: Failure isolation justifies the operational overhead of an independent service.

Problem 3 — Startup MVP Architecture

Scenario: You're building a SaaS project management tool with a team of 4 engineers. A tech lead insists on microservices from day one. Push back?

Solution: Yes — advocate strongly for a monolith. With 4 engineers, the operational overhead of microservices (separate CI/CD pipelines, inter-service contracts, distributed tracing, service discovery) will consume 30–40% of engineering bandwidth with zero user-facing benefit. A well-structured monolith with clear module boundaries (users, projects, tasks, billing as separate modules with defined interfaces) provides the same conceptual separation and can be extracted into services when pain points emerge — typically when team grows past ~15 engineers or when a specific module’s scaling or release cycle diverges. The cost of premature decomposition is a distributed monolith: all the complexity, none of the benefits.

Key trade-off stated: Operational cost of microservices only pays off at team/scale thresholds that don’t yet exist.

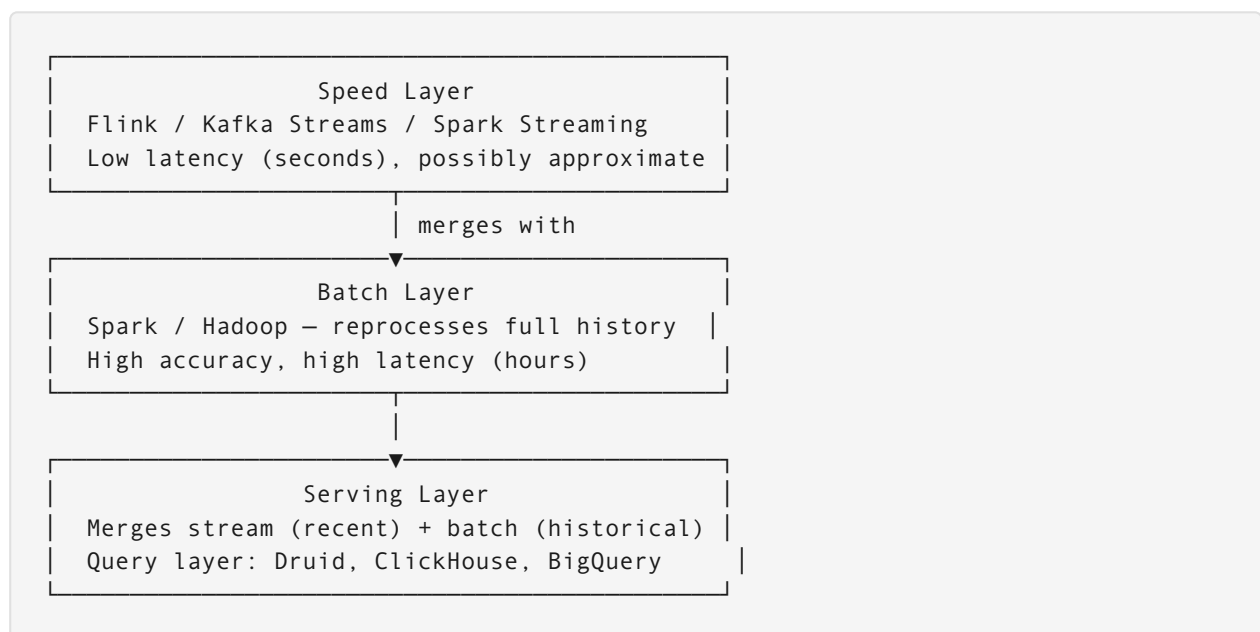
13.1.3 — Batch vs. Stream Processing

What It Is

Batch processing operates on large datasets accumulated at rest — run periodically (hourly, daily) with high accuracy but high latency. Stream processing operates on events in motion — continuous, low latency, but potentially approximate. Lambda Architecture combines both layers. Kappa Architecture simplifies to stream-only with replay capability.

Core Mechanics

Lambda Architecture:



Kappa Architecture: - Stream-only — eliminate the batch layer entirely. - Reprocess historical data by replaying the event log from the beginning (Kafka retention). - Single code path for both real-time and historical — eliminates the dual-codebase problem of Lambda. - Requires high-throughput, long-retention event log and the ability to replay at speed. - Higher engineering maturity required — replay must be idempotent and fast.

Trade-Off Matrix

Pattern	Latency	Accuracy	Complexity	Cost	Best For
Batch only	Hours	High — full dataset	Low	Low	End-of-day reports, ETL
Stream only	Seconds	Medium — windowed approximation	Medium	Medium	Fraud detection, alerting
Lambda	Seconds + Hours	High — batch corrects stream	High	High	Dashboards needing accuracy + recency
Kappa	Seconds	High (with replay)	Very High	Medium-High	Unified pipelines at scale

When / Where / How to Use

Batch: Payroll processing, end-of-day financial reconciliation, training ML models on historical data, large-scale ETL pipelines. Correctness matters more than latency.

Stream: Fraud detection (act within seconds of a suspicious transaction), real-time dashboards (live user counts), operational alerting (CPU spike → page), recommendation freshness (update “trending now” every minute).

Lambda: Business intelligence dashboards that must show both real-time data *and* accurate historical aggregates (e.g., revenue today vs. same day last year). Accept the operational cost of two codebases for the accuracy guarantee.

Kappa: When a single team owns the entire pipeline and can manage replay complexity. Works well when the event log (Kafka) already exists and historical reprocessing needs are frequent. Avoids the Lambda synchronisation problem.

Interview Design Problems

Problem 1 — Real-Time Fraud Detection

Scenario: A payment processor needs to detect fraudulent transactions within 200 ms of submission and decline them. Design the processing architecture.

Solution: Pure stream processing is required — batch cannot meet the 200 ms SLA. Use Kafka as the ingestion bus. A stateful stream processor (Flink) maintains rolling windows per card: transactions in the last 60 seconds, average spend per merchant category over 24 hours, geolocation velocity (card used in Singapore then New York within 1 hour = fraud signal). Flink writes per-card feature state to RocksDB (embedded, local storage). The fraud model scores each transaction in < 10 ms and publishes a DECLINE/ALLOW decision back to Kafka. A downstream batch job (daily Spark job) reprocesses all transactions against an improved fraud model to retroactively flag edge cases that the real-time model missed, updating account risk scores.

Key trade-off stated: Stream for sub-SLA decisions; batch for model retraining and retroactive correction — a partial Lambda pattern.

Problem 2 — Analytics Dashboard Accuracy

Scenario: A SaaS company's usage dashboard shows daily active users, feature usage, and revenue. The data team complains that the real-time stream numbers don't match the end-of-day batch report. How do you fix this?

Solution: This is the core problem Lambda Architecture solves. Implement a Lambda layer: the Speed Layer (Kafka + Flink) gives approximate real-time counts visible within 30 seconds. The Batch Layer (Spark on S3) runs every hour and reprocesses the full event log to produce accurate counts. The Serving Layer (ClickHouse) stores both batch results and stream increments; queries merge them — use batch data for hours > 1 hour old and stream data for the most recent hour. Users see a note "last hour figures are estimates, refreshed every minute" for transparency. Alternatively, migrate to Kappa — replay from Kafka on-demand to reconcile. But Kappa requires Kafka retention of 90+ days, which is a storage and operational cost decision.

Key trade-off stated: Lambda solves accuracy vs. recency; the cost is two codebases and a serving merge layer.

Problem 3 — Log Aggregation Pipeline

Scenario: Your infrastructure generates 500 GB of logs per day across 1,000 services. You need to support both real-time alerting (spike in 5xx errors) and historical queries (what errors occurred during last Tuesday's incident).

Solution: Use a Kappa-style unified pipeline. Ingest all logs into Kafka (retain for 7 days, sufficient for incident replay). A stream processor (Flink) consumes continuously: aggregates error rates per service per minute and writes to a time-series store (Prometheus / InfluxDB) for alerting. Simultaneously, a batch export job (Kafka Connect + S3 Sink) archives raw logs to S3 in Parquet format for long-term historical queries (Athena / Spark). For incident replay, re-consume from Kafka up to 7 days back; for older, query S3. The single Kafka log is the source of truth — no dual codebase, just two consumers with different SLAs.

Key trade-off stated: Kappa simplicity is achievable here because alerting (stream) and historical queries (S3 cold path) have naturally separate consumers; no serving-layer merge is needed.

13.1.4 — Push vs. Pull Architecture

What It Is

Push delivers updates from producer to consumer immediately upon availability. Pull has consumers fetch data from a source on demand or on a schedule. The choice affects latency, backpressure handling, coupling, and failure modes.

Core Mechanics

Push (Fan-out-on-write):

Producer → writes event → Message Queue / Push Notification → Consumer

Example: New tweet → Kafka → Fan-out service writes to all followers' feeds

Latency: near-instant delivery

Risk: if consumer is slow, queue grows (backpressure problem)

Failure: producer may push into the void if consumer is down

Pull (Fan-out-on-read):

Consumer → polls source → fetches latest data → processes

Example: Timeline read → query tweet store + follow graph → merge and sort

Latency: request time includes query + merge cost

Risk: thundering herd if many clients poll simultaneously
Benefit: consumer controls its own pace

Trade-Off Matrix

Aspect	Push	Pull
Latency	Near-immediate	Eventual — depends on poll interval
Backpressure	Producer overwhelms broker if consumer is slow	Consumer self-regulates pace
Failure Isolation	Poor — cascading failures if consumer down	Excellent — consumer retries independently
Coupling	Tight — producer must know consumers	Loose — consumers discover source independently
Implementation	Complex — delivery guarantees, dead-letter queues	Simple — polling loop
Fan-out Cost	Write time — expensive for large follower counts	Read time — expensive for complex merge queries
Freshness	Immediate	Bounded by poll interval

When / Where / How to Use

Choose Push When: - Real-time delivery is critical — chat messages, live notifications, stock tickers. - Consumer count is bounded and known — internal service-to-service events. - You have backpressure mechanisms (Kafka consumer groups, flow control).

Choose Pull When: - Consumer pace is unpredictable — batch jobs, analytics pipelines. - Consumers are external / untrusted — can't push to arbitrary clients. - Simplicity matters — a polling cron job is far simpler than a push notification system. - Fan-out count is enormous — pushing to 100M followers on write is prohibitively expensive.

Hybrid (Twitter/Instagram model): - Fan-out-on-write for users with < 10,000 followers (push to feed cache). - Fan-out-on-read for celebrities with > 1M followers (pull and merge at read time). - This controls write amplification while maintaining low-latency reads for most users.

Interview Design Problems

Problem 1 — Notification System at Scale

Scenario: Design a notification system for a social app with 500M users. Users receive notifications when someone likes, comments, or follows them. How do you handle delivery?

Solution: Use a push architecture with a fan-out queue. When User A likes User B's photo, write a like event to Kafka. A Notification Fan-out Service consumes the event and writes a notification record to User B's notification store (Cassandra — optimised for high write throughput, keyed by user_id). Push delivery to User B's mobile client via APNs/FCM (push notifications) if the device is offline, or via WebSocket if the app is open. Use pull for the notification inbox — when User B opens the notifications tab, the client pulls recent notifications from the Cassandra store. This avoids pushing a potentially large notification batch on app open. Backpressure is handled at the APNs/FCM layer — they queue and deliver based on device availability.

Key trade-off stated: Push for immediate delivery; pull for inbox reads — hybrid matches each access pattern.

Problem 2 — Configuration Management

Scenario: Your microservices need to pick up updated configuration (feature flags, rate limits) within 30 seconds of a change. Should the config service push updates or have services pull?

Solution: Pull with a short poll interval (15 seconds) is simpler and more resilient. Each service polls the config endpoint every 15 seconds. On change, all services will pick it up within 15–30 seconds — within the SLA. Push is more complex: requires a persistent connection (WebSocket / SSE / gRPC streaming) from every service instance to the config service, and a failure in the config service's push path means services miss updates silently. With pull, a temporary config service outage simply means services continue with their last known config — a safe failure mode. For large fleets (> 10,000 instances), use a tiered pull: service instances pull from local sidecar agents, which pull from the central config service — this reduces load on the central service.

Key trade-off stated: Pull's resilience and simplicity outweigh push's marginal latency improvement for config propagation.

Problem 3 — Live Sports Score Updates

Scenario: Design the score delivery system for a sports app. 10M users simultaneously watching a live game need score updates within 1 second of a goal being scored.

Solution: Push is mandatory — a 1-second SLA cannot be met by polling (would require polling every 200 ms × 10M clients = 50M requests/second on the backend). Use WebSockets for clients that are actively viewing a game. The backend publishes score events to a Pub/Sub system (Redis Pub/Sub or Kafka). A fleet of WebSocket gateway servers subscribes to the relevant game's topic and pushes updates to connected clients. This is a fan-out-on-push pattern: one event → one Kafka message → N gateway servers each push to their connected clients. No per-user write amplification because the fan-out happens at the WebSocket layer, not the message layer. For clients that go offline momentarily, reconnection fetches the latest score via a pull REST call and then re-subscribes.

Key trade-off stated: Latency SLA mandates push; scale of fan-out requires a gateway fan-out pattern rather than per-user messages.

13.1.5 — Synchronous vs. Asynchronous Communication

What It Is

Synchronous communication blocks the caller until a response is received (HTTP REST, gRPC unary). Asynchronous communication decouples sender and receiver — the sender continues immediately and handles the result later (message queues, event streams, callbacks). The choice directly affects user experience, system resilience, and failure propagation.

Core Mechanics

Synchronous Request:

```
Client —[request]→ Service —[process]→ Response → Client
```

▲
Client blocks here

Latency = Network RTT + Processing time

Failure = Client receives timeout or error; must retry or show error to user

Coupling: caller must know address of callee; callee must be available

Asynchronous Messaging:

```
Client —[message]→ Queue/Broker → Worker (eventually)
```

|
└ continues immediately

|
Result written to
result store / callback

Latency = hidden from caller; result arrives later

Failure: queue absorbs spikes; worker retries independently
Coupling: loose — producer and consumer are decoupled in time and space

Trade-Off Matrix

Dimension	Synchronous	Asynchronous
User-visible latency	High — user waits	Low — “we’ll notify you”
Caller complexity	Simple — request/response	Higher — must handle callbacks, polling, or events
Failure propagation	Tight — downstream failure surfaces to caller	Isolated — queue absorbs failures; worker retries
Throughput	Limited by slowest component in chain	High — queue acts as buffer for load spikes
Exactly-once semantics	Easier — single transaction	Harder — requires idempotent consumers and deduplication
Observability	Easy — single trace	Hard — distributed trace spans across queue boundaries
Order guarantees	Natural — sequential calls	Depends on queue (Kafka per-partition order; SQS no guarantee)

When / Where / How to Use

Choose Synchronous When: - The user needs an immediate result (login, search, checkout confirmation). - The operation must complete atomically before the response is sent. - Simplicity matters and the dependency graph is shallow. - Latency is critical and the downstream service is reliable and fast.

Choose Asynchronous When: - The operation is long-running (video encoding, report generation, bulk email). - The caller should not be blocked by downstream failures (payment notification vs. payment itself). - You need to absorb traffic spikes without dropping requests. - Multiple consumers need to react to the same event (event-driven architecture). - Retry logic should be handled by the system, not the caller.

Common Hybrid Pattern — Sync Front-End, Async Back-End:

User submits order → Sync: validate and reserve → Return "Order Placed" (202 Accepted)

↓

```
worker                                Async queue → payment worker → fulfilment worker → email
```

The user gets an immediate acknowledgment; slow back-end steps run asynchronously and the user is notified on completion.

Interview Design Problems

Problem 1 — Order Processing System

Scenario: An e-commerce checkout involves: (1) validate cart, (2) charge payment, (3) reduce inventory, (4) send confirmation email, (5) notify warehouse. Which steps are sync, which are async?

Solution: Sync steps (must complete before “Order Confirmed” is shown): validate cart (fast, local), charge payment (user must know if it succeeded). Return 200 with order ID only after these succeed.

Async steps (fire-and-forget after payment succeeds): reduce inventory (can be eventual — oversell is handled by saga compensation), send confirmation email (latency doesn’t affect UX), notify warehouse (warehouse doesn’t need sub-second notification). Publish an `OrderPlaced` event to Kafka after payment succeeds. Three independent consumers process it: `InventoryService`, `EmailService`, `WarehouseService`. If the email service is down, Kafka retains the event and the email is sent when the service recovers — no user impact.

Key trade-off stated: Sync for user-blocking steps; async for everything that doesn’t affect the immediate user response.

Problem 2 — Video Upload and Processing

Scenario: Users upload videos to a platform. After upload, the system must transcode to 5 resolutions, generate a thumbnail, and run a content moderation check. How do you architect this?

Solution: Upload is synchronous — the user needs to know their upload succeeded. After the upload is stored in S3, immediately return “Upload successful, processing...” (202 Accepted). The upload handler publishes a `VideoUploaded` event to SQS. Three independent async workers consume it: `TranscodeWorker` (fans out to 5 sub-tasks per resolution), `ThumbnailWorker`, `ModerationWorker`. Each worker is independently scalable. Transcoding is the slowest (minutes) — async is essential here; no user should wait synchronously for transcoding. The UI polls a `/status/{videoId}` endpoint every 10 seconds. Once all workers complete, the video is marked `READY`. Failed workers retry with exponential backoff; after 3 failures, the video is marked `PROCESSING_FAILED` and the user is notified by email.

Key trade-off stated: Async hides processing latency and enables independent worker scaling and failure handling.

Problem 3 — Microservice Dependency Failures

Scenario: Service A calls Service B synchronously. Service B begins experiencing latency spikes (5-second responses). Service A's threads are exhausted waiting. The entire platform degrades. How do you fix this architecturally?

Solution: Apply two patterns: first, add a circuit breaker to the synchronous call — if Service B's error rate or latency exceeds a threshold, the circuit opens and Service A fails fast (returns a cached or degraded response instead of waiting). Second, evaluate whether the Service A → Service B call needs to be synchronous at all. If it does not need to block the user response (e.g., it's enriching a background record), convert it to async: Service A publishes a request event, Service B processes and publishes a response event, and Service A consumes the response. This decouples their latency profiles entirely. The root cause fix (Service B performance) still needs to happen, but the architectural fix prevents cascading failures in the interim.

Key trade-off stated: Synchronous calls create tight coupling in failure modes; async decoupling or circuit breakers restore resilience.

13.1.6 — Summary: Decision Framework

Use this table as a quick-reference when structuring your answer in a system design interview. Always state the trade-off explicitly — interviewers are evaluating your reasoning, not just your conclusion.

Decision	Default Choice	Override When
Vertical vs. Horizontal	Horizontal for stateless	Latency SLA < 1ms, or stateful without sharding budget
Monolith vs. Microservices	Monolith for < 10 engineers	Team > 15, distinct scaling/failure domains emerge
Batch vs. Stream	Batch for accuracy	Latency SLA < minutes, or real-time alerting needed
Lambda vs. Kappa	Kappa for simplicity	Accuracy + recency required simultaneously at scale
Push vs. Pull	Pull for resilience	

Decision	Default Choice	Override When
		Latency SLA < seconds, or fan-out to bounded consumers
Sync vs. Async	Sync for simplicity	Long-running ops, spike absorption, failure isolation needed

13.1.7 — Consistency vs. Availability (CAP Theorem)

Math-heavy section. Give extra weight to worked numeric examples and proof-by-contradiction style explanations for correctness arguments.

What It Is

The CAP Theorem (Brewer, 2000; formally proved by Gilbert & Lynch, 2002) states that a distributed system can guarantee at most **two** of the following three properties simultaneously:

- **Consistency (C):** Every read receives the most recent write or an error. All nodes see the same data at the same time.
- **Availability (A):** Every request receives a (non-error) response — though it may not contain the most recent data.
- **Partition Tolerance (P):** The system continues operating despite an arbitrary number of messages being dropped or delayed by the network between nodes.

The practical consequence: In any real distributed system, network partitions *will* occur (hardware fails, cables get cut, data centres lose connectivity). Therefore **P is not optional** — you must tolerate partitions. The real choice is: *during a partition, do you sacrifice C or A?*

Proof-by-Contradiction: Why You Cannot Have All Three

Assume a two-node distributed system (Node 1, Node 2) that claims C + A + P.

Setup:

Node 1 and Node 2 both store variable $X = 0$.

A network partition occurs: Node 1 and Node 2 cannot communicate.

Step 1: Client writes $X = 1$ to Node 1.

Node 1 accepts the write (Availability — must not refuse).

Node 1 cannot propagate to Node 2 (Partition).

```

Step 2: Client reads X from Node 2.
        Node 2 must respond (Availability — must not refuse).
        Node 2 still has X = 0 (has not received the write).
        Node 2 returns X = 0.

Result: The system returned a stale value.
        This violates Consistency (the read did not reflect the most recent
        write).

Contradiction: The system cannot be simultaneously C + A + P. ■

```

Therefore: you must choose **CP** (sacrifice availability — return an error during partition) or **AP** (sacrifice consistency — return potentially stale data).

PACELC Extension

CAP only addresses behaviour *during* a partition. PACELC extends this to normal operation:

```

IF Partition:
    choose between Availability (A) vs. Consistency (C)
ELSE (normal operation):
    choose between Latency (L) vs. Consistency (C)

```

This is more practically useful. Even without a partition, there is a latency cost to strong consistency — you must wait for acknowledgement from multiple replicas before returning a result.

Consistency Models (Weakest → Strongest)

Model	Guarantee	Example System	Latency Impact
Eventual Consistency	All replicas converge <i>eventually</i>	DynamoDB (default), Cassandra	Lowest
Monotonic Read	A client never reads older data than it previously read	Cassandra with session consistency	Low
Read-Your-Writes	After a write, the same client always reads the new value	DynamoDB strongly consistent reads	Low-Medium
Causal Consistency	Causally related operations appear in order to all nodes	MongoDB causal sessions	Medium
Sequential Consistency	All operations appear in some	ZooKeeper	Medium-High

Model	Guarantee	Example System	Latency Impact
	sequential order consistent with program order		
Linearisability (Strong)	Operations appear instantaneous; reads always reflect the latest write	etcd, Google Spanner	Highest

Worked Numeric Example: Replication Quorums

A Cassandra cluster has **N = 5** replicas. You configure: - **W** = write quorum (how many replicas must acknowledge a write) - **R** = read quorum (how many replicas must respond to a read)

Rule for strong consistency: $W + R > N$

Example 1 – Strong Consistency (W=3, R=3):

$W + R = 6 > N = 5$ ✓ Strong consistency guaranteed.

Write latency = time for 3rd-slowest of 5 replicas to acknowledge.

Read latency = time for 3rd-slowest of 5 replicas to respond.

Example 2 – High Availability (W=1, R=1):

$W + R = 2 \leq N = 5$ ✗ No consistency guarantee.

A read may hit a replica that hasn't received the latest write.

Write latency = time for fastest replica. Fast, but stale reads possible.

Example 3 – Typical Balanced Config (W=2, R=2):

$W + R = 4 \leq N = 5$ ✗ Weak consistency.

Still faster than Example 1. Used when "usually consistent" is acceptable.

Example 4 – Read-Heavy Optimised (W=3, R=1): [Hinted Handoff pattern]

$W + R = 4 \leq N = 5$ ✗ No guarantee.

But: 3 replicas written means a random read has $3/5 = 60\%$ chance of hitting a consistent replica. Useful for non-critical read-heavy workloads.

Key insight: There is a direct latency cost to higher W and R values — you must wait for more replicas. Strong consistency doubles or triples tail latency under load.

Trade-Off Matrix

System	CAP Choice	PACELC	Use Case
Cassandra	AP	EL (Eventual, Low latency)	High-write, eventual-ok (analytics, IoT)

System	CAP Choice	PACELC	Use Case
DynamoDB (default)	AP	EL	Shopping carts, session data
DynamoDB (strongly consistent)	CP	EC	Financial account balances
etcd / ZooKeeper	CP	HC (High consistency)	Leader election, config management
Google Spanner	CP	EC (with TrueTime)	Global financial transactions
MongoDB (replica set)	CP (w: majority)	EC	General purpose with consistency needs
Couchbase	AP	EL	Mobile sync, offline-first apps

When / Where / How to Use

Choose AP (Eventual Consistency) When: - Stale reads are tolerable and the domain naturally reconciles (shopping cart — user can see slightly stale item count, final count computed at checkout). - Write throughput must not be blocked by replica lag. - Global distribution makes synchronous replication latency prohibitive (US → EU synchronous write = 100+ ms round trip). - The application handles conflicts at the application layer (CRDT, last-write-wins).

Choose CP (Strong Consistency) When: - Correctness is non-negotiable: financial balances, inventory counters, distributed locks. - The application cannot handle stale reads (a user must always see the money they just deposited). - You are implementing coordination primitives (leader election, distributed mutex).

Interview Design Problems

Problem 1 — Bank Account Balance

Scenario: You are designing the data store for bank account balances. A user transfers \$100 from Account A to Account B. What consistency model is required and why?

Solution: Linearisability (strong consistency) is required. Consider the failure mode under eventual consistency: after the debit of Account A completes, a network partition prevents the credit to Account

B from propagating immediately. A read of Account B's balance returns the pre-credit value. The \$100 appears to have vanished — a correctness violation that is unacceptable in finance.

Implement this with a CP system: use a relational database with ACID transactions (PostgreSQL), or a distributed system configured for strong consistency (DynamoDB strongly consistent reads + conditional writes, or Google Spanner). The $W+R > N$ quorum rule must hold. Accept higher latency as the cost: a 50 ms transfer response time is acceptable; a \$100 disappearing act is not.

Proof-by-contradiction argument for the interview: “If I use eventual consistency, then during a partition, a read of Account B can return the pre-transfer balance. This contradicts the invariant that the sum of all account balances must remain constant. Therefore eventual consistency violates our correctness requirement.”

Problem 2 — Global Shopping Cart

Scenario: Amazon's shopping cart is served from data centres in US-East, EU-West, and AP-Southeast. A user in Singapore adds an item. Milliseconds later they refresh from a different device. Should this use CP or AP?

Solution: AP (eventual consistency) is the right choice, and Amazon's Dynamo paper (2007) describes exactly this decision. The cost of a stale read (cart shows 2 items instead of 3 for half a second) is far lower than the cost of an unavailable cart (user cannot add items during a US-East outage). Strong consistency would require synchronising across three regions before confirming the add — adding 150–300 ms of cross-region latency to every cart operation.

Conflict resolution uses a “merge” strategy: if two devices add different items simultaneously during a partition, both items appear in the cart on reconciliation (union merge). Duplicate adds are idempotent. The invariant “all items the user added should appear” is eventually satisfied.

Numeric trade-off: Synchronous cross-region replication US-East ↔ AP-Southeast $\approx$ 200 ms RTT. Async local write $\approx$ 5 ms. For a shopping cart, the 40× latency improvement of AP is worth the < 1 -second eventual consistency window.

Problem 3 — Distributed Lock for Job Scheduling

Scenario: A job scheduler must ensure exactly one worker runs a given cron job at a time across 50 worker nodes. What consistency model is required and how do you implement it?

Solution: CP / Linearisability is required. A distributed lock by definition requires that all nodes agree on who holds the lock. Under eventual consistency, two nodes could simultaneously believe they

hold the lock — leading to duplicate job execution (double-sending emails, double-charging customers).

Implementation: use etcd or ZooKeeper, both of which implement linearisable consensus (Raft / Zab). The worker attempting to acquire the lock writes a key with a TTL using a Compare-And-Swap (CAS) operation: “set key=job_X_lock to worker_42 only if key does not currently exist.” Because etcd is CP, this CAS is atomic and linearisable — only one writer will succeed across all 50 workers. The TTL ensures the lock is released if the worker crashes. Workers that fail to acquire the lock wait and retry.

Why not AP here: “If I used an AP system like Cassandra with W=1, two workers could both succeed at writing the lock key due to replica divergence during a partition. Both would proceed to execute the job. This violates the exactly-once invariant the scheduler requires.”

13.1.8 — Latency vs. Throughput

***Math-heavy section.** Give extra weight to worked numeric examples and proof-by-contradiction style explanations for correctness arguments.*

What It Is

Latency is the time taken to complete a single operation (milliseconds per request). **Throughput** is the number of operations completed per unit of time (requests per second). These are related but often in tension — optimising one can degrade the other. Understanding the relationship, and where the system’s actual bottleneck lies, is essential for senior-level system design.

Core Mechanics

Little’s Law — the fundamental relationship between latency, throughput, and concurrency:

$$L = \lambda \times W$$

Where:

L = average number of requests in the system (concurrency / queue depth)

λ = throughput (requests per second arriving / being processed)

W = average latency (seconds per request)

Rearranged:

Throughput (λ) = L / W

Latency (W) = L / λ

Worked Example:

A web service has:

- 200 concurrent requests in flight at any time ($L = 200$)
- Average latency of 100 ms per request ($W = 0.1$ s)

Throughput = $L / W = 200 / 0.1 = 2,000$ RPS

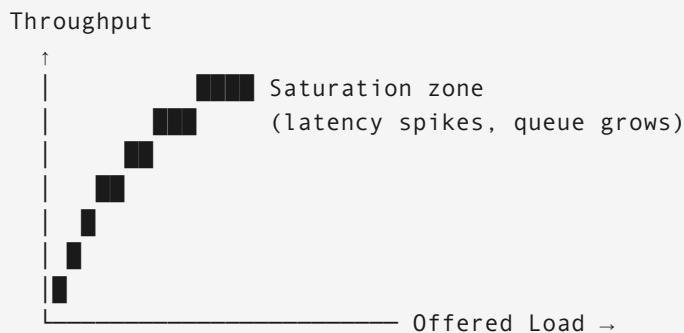
Now you optimise the handler and reduce latency to 50 ms ($W = 0.05$ s):

Throughput = $200 / 0.05 = 4,000$ RPS (doubled without adding hardware)

Alternatively, you add more concurrency ($L = 400$) without changing latency ($W = 0.1$ s):

Throughput = $400 / 0.1 = 4,000$ RPS (same result, but via capacity not efficiency)

The Latency-Throughput Curve:



At low load: latency is stable and low; throughput scales linearly.

At ~70% capacity: latency begins to increase (queue forms).

At saturation: latency spikes exponentially; throughput plateaus or drops (congestion collapse).

Amdahl's Law — limits of parallelism for throughput improvement:

$$\text{Speedup} = 1 / (S + (1 - S) / N)$$

Where:

S = fraction of work that is inherently serial (cannot be parallelised)

N = number of parallel processors / workers

Example: A task is 20% serial ($S = 0.2$), 80% parallelisable.

$N = 1$: Speedup = $1.0\times$

$N = 4$: Speedup = $1 / (0.2 + 0.8/4) = 1 / 0.4 = 2.5\times$

$N = 10$: Speedup = $1 / (0.2 + 0.8/10) = 1 / 0.28 = 3.57\times$

$N = \infty$: Speedup = $1 / 0.2 = 5\times$ (hard ceiling)

Key insight: Even infinite parallelism cannot overcome a 20% serial bottleneck.
The serial fraction dominates at scale.

Optimisation Levers

Lever	Effect on Latency	Effect on Throughput	Cost
Caching (in-memory)	↓↓ (10–100× reduction)	↑↑ (eliminates DB round trip)	Memory cost; staleness risk
Connection pooling	↓ (eliminates TCP handshake)	↑↑ (reuse connections)	Config complexity
Batching	↑ (individual request waits)	↑↑ (amortises fixed overhead)	Higher latency per item
Async I/O	— (same wall time)	↑↑ (thread not blocked)	Code complexity
Sharding / partitioning	↓ (smaller dataset per node)	↑↑ (parallel processing)	Operational complexity
Read replicas	↓ (for reads)	↑ (distribute read load)	Replication lag risk
CDN / Edge caching	↓↓ (geo-proximity)	↑ (offload origin)	Cache invalidation
Compression	↑ (CPU cost to compress)	↑ (less data over wire)	CPU trade-off
Indexes	↓ (faster query)	↑ (less I/O per query)	Write overhead; storage

Latency Numbers Every Engineer Should Know (2024 approximate)

Operation	Latency	Notes
L1 cache reference	~0.5 ns	
Branch misprediction	~5 ns	
L2 cache reference	~7 ns	
Mutex lock/unlock	~25 ns	
Main memory (RAM) reference	~100 ns	200× slower than L1

Compress 1 KB (snappy)	~3,000 ns	3 µs
Send 1 KB over 1 Gbps network	~10,000 ns	10 µs
SSD random read (NVMe)	~100,000 ns	100 µs; 1,000× slower than RAM
Read 1 MB sequentially (RAM)	~250,000 ns	250 µs
Round-trip within same DC	~500,000 ns	0.5 ms
SSD sequential read (1 MB)	~1,000,000 ns	1 ms
HDD random seek	~10,000,000 ns	10 ms; 100× slower than SSD
Round-trip US → EU (TCP)	~100,000,000 ns	100 ms
Round-trip US → Australia	~200,000,000 ns	200 ms

Interview application of these numbers: - A synchronous call chain of 10 microservices, each adding 0.5 ms network RTT = **5 ms overhead minimum** before any business logic runs. - Reading 1 MB from HDD during a request = **10 ms** — instantly blows a 5 ms SLA. Cache it. - A cross-region synchronous write (US → EU) = **100 ms** — unacceptable for interactive UX. Use async replication.

Trade-Off Matrix

Optimisation Goal	Technique	Latency	Throughput	Risk
Lower p99 latency	Connection pooling, caching hot paths	↓↓	↑	Stale cache, pool exhaustion
Higher throughput	Batching, async I/O, horizontal scale	↑ per item	↑↑	Head-of-line blocking in batches
Both	Sharding + caching	↓	↑↑	Operational complexity
Handle spikes	Queue + async workers	↑ (queuing delay)	Stable	Latency unpredictable under load

When / Where / How to Use

Optimise Latency When: - User-interactive flows: search, checkout, login — users perceive > 200 ms as “slow” and > 1 s as broken. - Latency SLA is contractual (p99 < 100 ms in an SLA). - Downstream callers are synchronous and blocked waiting.

Optimise Throughput When: - Batch workloads: log processing, ETL, report generation — latency per item is irrelevant; total completion time matters. - Cost efficiency: more work per machine = lower cost per operation. - The bottleneck is not user-facing (background job pipelines).

Identify Your Bottleneck First (USE Method):

For every resource (CPU, memory, disk I/O, network):
Utilisation: Is it near 100%? (top, vmstat, iostat)
Saturation: Is there a queue forming? (run queue length, disk queue depth)
Errors: Are operations failing? (TCP retransmits, disk errors)

The first resource to saturate is your bottleneck.
Optimising anything else has diminishing returns until the bottleneck is addressed.

Interview Design Problems

Problem 1 — API Response Time Degrading Under Load

Scenario: Your API serves 500 RPS normally at p99 = 50 ms. When load increases to 1,500 RPS, p99 jumps to 2,000 ms. Diagnose and fix.

Solution: Apply Little's Law to diagnose. At 500 RPS with p99 = 50 ms, concurrency $L = \lambda \times W = 500 \times 0.05 = 25$ requests in flight. At 1,500 RPS with p99 = 2,000 ms, $L = 1,500 \times 2.0 = 3,000$ — a 120x increase in queue depth. This is textbook saturation: the system has hit its concurrency ceiling and requests are queueing. The latency spike is caused by queueing delay, not processing time.

Fix options in priority order: 1. Find the saturated resource (CPU? DB connections? downstream API?). Most likely: database connection pool exhausted. Increase pool size or add a read replica. 2. Add a rate limiter / load shedder at 1,200 RPS to return 503 rather than queueing indefinitely — this protects p99 for requests that do get through. 3. Add horizontal API servers behind the load balancer to increase L ceiling. 4. Cache the most frequent read queries to reduce DB pressure.

Numeric validation: After adding a read replica and increasing connection pool from 20 → 80, at 1,500 RPS the pool is no longer exhausted. Queue drains. Latency returns to ~60 ms (slight increase due to higher load, but no saturation cliff).

Problem 2 — Batch Pipeline Throughput

Scenario: A daily ETL pipeline processes 10 TB of log data. Currently it runs on a single node and takes 18 hours. You need it to complete in under 2 hours. What's your approach?

Solution: Apply Amdahl's Law. First, profile the pipeline to find the serial fraction. Assume 5% of the pipeline is serial (file listing, final merge, schema validation) — $S = 0.05$.

Maximum speedup = $1 / S = 1 / 0.05 = 20\times$

Required speedup = 18 hours / 2 hours = 9x

$9\times < 20\times$ — achievable within Amdahl's ceiling.

Workers needed: $9 = 1 / (0.05 + 0.95/N)$
 $0.05 + 0.95/N = 1/9 = 0.111$
 $0.95/N = 0.061$
 $N = 0.95 / 0.061 \approx 15.6 \rightarrow 16$ workers required.

Implement with Spark on 16 executor nodes, partitioning the 10 TB across 16 parallel tasks. The serial phases (job planning, final output write) will still take their fixed time, but the 95% parallelisable processing phase completes in 1/16th the time. Run on spot/preemptible instances to control cost — Spark handles node loss via task retry.

Key insight stated: Profile the serial fraction first. Adding 100 workers when $S = 0.3$ only gives 3.2× speedup — a waste of cost and complexity.

Problem 3 — Cache vs. Database Latency

Scenario: A product detail page makes 3 database queries (product info, inventory, reviews), each taking 20 ms. Total page latency = 60 ms (sequential) or 20 ms (parallel). Target is $p99 < 5$ ms. How do you get there?

Solution: Even with parallel execution, 20 ms database latency exceeds the 5 ms SLA. The solution is caching. Introduce Redis (in-memory, ~0.2 ms read latency) in front of each query:

Cache hit path: 3 parallel Redis reads $\times 0.2$ ms = 0.2 ms wall time $\checkmark < 5$ ms
Cache miss path: Redis miss $\rightarrow$ DB query $\rightarrow$ write to Redis $\rightarrow 20$ ms $\times > 5$ ms

Cache hit rate needed to meet $p99 < 5$ ms:

If 1% of requests are cache misses (hit rate = 99%):

$p99 = 99\text{th percentile} \approx \text{cache hit latency} = 0.2$ ms $\checkmark$

(Only 1 in 100 requests hits the miss path — those requests are slow, but p99 is fine)

If 5% are cache misses (hit rate = 95%):

$p95 = \text{cache hit}$. $p99$ may be a miss. Need to check actual distribution.

Set cache TTL based on data staleness tolerance: product info (TTL = 1 hour), inventory (TTL = 30 seconds — must be relatively fresh to avoid overselling), reviews (TTL = 5 minutes). Use cache-aside pattern: application checks Redis $\rightarrow$ miss $\rightarrow$ query DB $\rightarrow$ populate Redis. For inventory, use write-through cache: every inventory update writes to both DB and Redis atomically, keeping cache always fresh.

13.1.9 — Cost vs. Reliability

What It Is

Reliability is the probability that a system performs its required function without failure over a given time period. Cost is the financial expense of building, running, and maintaining that system. Higher reliability almost always requires redundancy — and redundancy costs money. The trade-off is quantifying *how much reliability you actually need*, because the last 1% of reliability is exponentially more expensive than the first 99%.

Core Mechanics

Availability Calculation:

Availability = Uptime / (Uptime + Downtime)

Expressed as "nines":

99% ("two nines") = 87.6 hours downtime/year
99.9% ("three nines") = 8.76 hours downtime/year
99.99% ("four nines") = 52.6 minutes downtime/year
99.999% ("five nines") = 5.26 minutes downtime/year

Cost to go from 99% → 99.9%: ~2–3× infrastructure cost

Cost to go from 99.9% → 99.99%: ~5–10× infrastructure cost

Cost to go from 99.99% → 99.999%: ~20–50× infrastructure cost

Serial vs. Parallel Reliability:

Components in SERIES (all must work):

System availability = $A_1 \times A_2 \times A_3 \times \dots$

Example: 3 services, each 99.9% available:

System = $0.999 \times 0.999 \times 0.999 = 0.997 = 99.7\%$

(Each dependency in a call chain degrades overall availability)

Components in PARALLEL (any one can serve the request):

System unavailability = $(1 - A_1) \times (1 - A_2) \times \dots$

System availability = $1 - (1 - A_1) \times (1 - A_2)$

Example: 2 replicas, each 99.9% available:

Unavailability = $0.001 \times 0.001 = 0.000001$

System = $1 - 0.000001 = 99.99999\%$ — SIX nines from two three-nine components!

(Redundancy in parallel dramatically improves availability)

Worked Cost vs. Reliability Example:

Scenario: A payments API, \$50,000 revenue/hour.

Downtime cost:

99% availability → 87.6 hrs/yr downtime × \$50K = \$4,380,000/yr lost
99.9% availability → 8.76 hrs/yr × \$50K = \$438,000/yr lost
99.99% availability → 0.876 hrs/yr × \$50K = \$43,800/yr lost

Infrastructure cost (rough order of magnitude):

99% → single-region, single AZ → \$50,000/yr infra
99.9% → multi-AZ, active-active → \$120,000/yr infra (+\$70K)
99.99% → multi-region, active-active → \$400,000/yr infra (+\$280K)

ROI analysis:

99% → 99.9% upgrade:

Saves \$3,942,000 in downtime, costs \$70,000 extra infra → ROI = 56× ✓ Do it.

99.9% → 99.99% upgrade:

Saves \$394,200 in downtime, costs \$280,000 extra infra → ROI = 1.4× ✓

Marginal.

99.99% → 99.999%:

Saves \$39,420 in downtime, costs ~\$800,000 extra infra → ROI = 0.05× x Not worth it.

Reliability Patterns and Their Cost

Pattern	Availability Gain	Cost Impact	Complexity
Multi-AZ deployment	99% → 99.99% (within region)	~2× compute cost	Low — managed by cloud
Multi-region active-active	Adds resilience to regional failures	~3–4× compute cost + data replication	High
Read replicas	Improves read availability; no write HA	+20–50% DB cost	Low
Multi-master DB	Improves write availability	~3× DB cost + conflict resolution	Very High
Circuit breakers	Prevents cascading failures	Negligible cost	Medium
Bulkheads	Isolates failure blast radius	+10–20% resource overhead	Medium
Chaos engineering	Validates reliability assumptions	Engineering time cost	High (culture change)
Hot standby	Fast failover (< 30 s RTO)		Medium

Pattern	Availability Gain	Cost Impact	Complexity
		~2x cost (standby is idle)	
Warm standby	Moderate failover (minutes)	~1.3x cost	Low
Cold standby	Slow failover (hours)	~1.05x cost	Low

RTO and RPO definitions:

RTO (Recovery Time Objective): Maximum acceptable downtime after a failure.
RPO (Recovery Point Objective): Maximum acceptable data loss (how old can the backup/replica be when we fail over?).

More aggressive RTO/RPO = more expensive:

RTO = 0s, RPO = 0s: Synchronous multi-region active-active → Very expensive
RTO = 5m, RPO = 1m: Hot standby with synchronous replication → Expensive
RTO = 1h, RPO = 1h: Warm standby with async replication → Moderate
RTO = 24h, RPO = 24h: Daily backup restore → Cheap

When / Where / How to Use

Invest heavily in reliability when: - Revenue loss per hour of downtime exceeds the annualised cost of redundancy (use the ROI calculation above). - Regulatory requirements mandate SLAs (financial services, healthcare, government). - Reputation damage from downtime is asymmetric (a single outage loses years of customer trust). - Safety is involved (medical devices, aviation, industrial control systems).

Accept lower reliability when: - Internal tooling or developer-facing systems where downtime is inconvenient but not costly. - Early-stage products where simplicity of architecture enables faster iteration. - Non-critical background jobs where retry tomorrow is acceptable. - Cost constraints are tight and the product hasn't proven revenue yet.

Interview Design Problems

Problem 1 — SLA Definition for a Payment Service

Scenario: You are the lead architect for a new payment processing service. Your CTO asks: "What SLA should we commit to, and what will it cost?" Walk through your reasoning.

Solution: Start with the business impact of downtime, not the technical target.

Payment services typically process transactions at, say, \$200,000/hour in GMV. Each hour of downtime = \$200,000 in lost transactions plus reputational damage. Calculate the break-even point for each tier of reliability investment:

99.9% → 99.99% upgrade:
Saves ~8 hrs downtime/year × \$200K = \$1.6M/year in prevented losses.
Multi-region active-active cost delta: ~\$300K/year in infra.
ROI = 5.3× → justify the investment.

99.99% → 99.999%:
Saves ~47 min/year × \$200K/hr = \$156K/year.
Cost delta for five-nines: ~\$1M/year.
ROI = 0.15× → not economically justified unless regulatory.

Recommend: commit to **99.99%** SLA. Implement via multi-AZ active-active within a primary region (handles AZ failures, ~70% of incidents) plus a warm standby in a second region (handles regional failures, ~5% of incidents) with RPO = 1 minute (synchronous replication within region, async cross-region).

State the trade-off explicitly: “Five-nines would require synchronous multi-region writes, adding 100+ ms latency to every transaction globally. That latency degradation harms user experience more than the 5 extra minutes of annual downtime it prevents.”

Problem 2 — Database Backup Strategy

Scenario: An early-stage startup’s database is currently backed up once daily to S3. The CTO asks whether this is sufficient. The product has 10,000 users and generates \$5,000/month revenue. What do you recommend?

Solution: Apply RPO analysis. A daily backup means RPO = 24 hours — in the worst case, you lose an entire day of data. For 10,000 users generating \$5K/month = ~\$167/day, losing one day of data means \$167 in direct revenue loss plus user trust damage. The more significant risk is: what data do users expect to be durable? If it’s financial transactions or user-created content, one day of data loss may drive user churn worth far more than \$167.

Recommendation: Upgrade to continuous WAL archiving (PostgreSQL) or point-in-time recovery (AWS RDS automated backups). Cost: ~\$10–30/month for WAL storage. This reduces RPO from 24 hours to ~5 minutes at negligible cost relative to the risk.

Do not yet invest in multi-region: at \$5K/month revenue, the ROI of multi-region (\$500+/month) is negative. The startup should accept higher RTO (a few hours to restore from backup) and focus RPO improvement (5-minute data loss window) as the higher-priority risk.

Numeric framing: Backup cost upgrade = \$20/month. Expected value of data loss prevented (assuming 1 outage/year, 10-hour average, \$167/day): $(10/24) \times \$167 = \70 . ROI = 3.5x over raw revenue, and far higher when user trust is factored in. Do it.

Problem 3 — Cascading Failure Prevention

Scenario: Your system has 8 microservices all calling a shared Recommendation Service. The Recommendation Service slows down under load. Within 2 minutes, all 8 services are saturated waiting for recommendations, and the entire platform goes down. How do you fix this architecturally?

Solution: This is a cascading failure caused by synchronous coupling without isolation. Three patterns address it in layers:

Layer 1 — Circuit Breaker: Each caller wraps its call to the Recommendation Service in a circuit breaker (Hystrix / Resilience4j). When the Recommendation Service's error rate exceeds 50% or latency exceeds 500 ms, the circuit opens. Callers immediately receive a fallback response (empty recommendations, cached results, or "no recommendations available") instead of waiting. This stops the saturation of caller thread pools. Cost: negligible. Benefit: prevents cascading failure.

Layer 2 — Bulkhead: Give each of the 8 caller services an independent, bounded thread pool for Recommendation Service calls. If the pool is exhausted (Recommendation is slow), only that pool is affected — the calling service continues processing all other work. The bulkhead prevents one slow dependency from consuming all of a service's resources. Cost: ~10% resource overhead per service.

Layer 3 — Async Degradation: For non-critical recommendation features (e.g., "Users like you also bought"), make the call asynchronous. Fetch recommendations in the background; return the page immediately without them. If recommendations arrive within 200 ms, inject them. If not, render without. Cost: architectural refactor, medium effort. Benefit: eliminates synchronous dependency entirely for non-critical features.

Reliability calculation: Before fix — any Recommendation Service degradation cascades to 100% of platform. After fix — Recommendation Service outage degrades recommendation features only; all other platform functionality remains at full availability. System availability goes from dependent on the Recommendation Service (its outage = full outage) to independent (its outage = partial feature degradation).

13.1.10 — Updated Summary: Complete Decision Framework

Use this table as a quick-reference across all trade-off dimensions covered in this section.

Decision	Default Choice	Override When	Key Signal
Vertical vs. Horizontal	Horizontal for stateless	Latency SLA < 1 ms, or tightly-coupled stateful	Can you make the service stateless?
Monolith vs. Microservices	Monolith for < 10 engineers	Team > 15, distinct scaling or failure domains	Conway's Law: does the org map to the desired architecture?
Batch vs. Stream	Batch for accuracy	Latency SLA < minutes, real-time alerting needed	What is the user-acceptable lag?
Lambda vs. Kappa	Kappa for simplicity	Accuracy + recency simultaneously required at scale	Can you afford 90-day Kafka retention for replay?
Push vs. Pull	Pull for resilience	Latency SLA < seconds, bounded consumer count	How large is the fan-out?
Sync vs. Async	Sync for simplicity	Long-running ops, spike absorption, failure isolation	Does the user need to wait for this result?
CP vs. AP	AP (eventual) for availability	Financial, locking, or correctness-critical data	What is the business cost of a stale read?
Latency vs. Throughput	Optimise latency for interactive	Batch/background jobs: optimise throughput	Who is waiting — a user or a cron job?
Cost vs. Reliability	Match reliability to revenue impact	Regulatory mandates or safety requirements	Calculate ROI of each reliability tier explicitly

Interview Tip: When asked “how would you scale X?”, always clarify: *What is the bottleneck? (CPU, memory, I/O, network, cost), What are the constraints? (latency SLA, consistency requirements, team size, budget), and What failure modes are acceptable? Only then apply the framework above. The best answers reason through trade-offs out loud rather than jumping to a solution. For consistency and reliability questions specifically, quantify the cost of failure before recommending a solution — interviewers at senior levels reward the engineer who says “let me calculate whether that reliability tier is worth the cost” over the one who reflexively answers “five nines.”*